3 A computer game requires users to travel around a world to find and open treasure chests. Each treasure chest has a mathematics question inside. The user enters the answer. The number of points awarded depends on the number of attempts before the user gives the correct answer.

The program will be created using object-oriented programming (OOP).

The following class diagram describes the class `TreasureChest`.

TreasureChest	

`question : STRING`
`answer : INTEGER`
`points : INTEGER`
	`// stores the question`
	`// stores the answer`
	`// stores the maximum possible number of points available for this chest`

`constructor()`
 	
`getQuestion()`
 	
`checkAnswer()`
 	
`getPoints()`
	`// takes question, answer and points as parameters and creates an instance of an object`
	
	`// returns the question`
	
	`// takes the user's answer as a parameter and returns True if it is correct, otherwise returns False`
	
	`// takes the number of attempts as a parameter and returns the number of points awarded`

(a) Create a new program.

Write program code to declare the class `TreasureChest`.

Do **not** write any other methods.

The attributes are private.

If you are using the Python programming language, include attribute declarations using comments.

Save your program as **question3**. Copy and paste the program code into **part 3(a)** in the evidence document.

[5]

(b) The text file `TreasureChestData.txt` stores data for five questions, in the order of question, answer, points.

For example, the first three lines of the file are for the first question:

`2*2` question
`4` answer
`10` points

Write program code for the procedure, `readData()` to:

- read each question, answer and points from the text file
- create an object of type `TreasureChest` for each question
- declare an array named `arrayTreasure` of type `TreasureChest`
- append each object to the array
- use exception handling to output an appropriate message if the file is not found.

Save your program. Copy and paste the program code into **part 3(b)** in the evidence document.

[8]

(c) The main program repeats each question until the user inputs the correct answer. The number of points awarded depends on the number of attempts before the user gives the correct answer.

(i) The class `TreasureChest` has a method `getQuestion()` that returns the question.

Write the method `getQuestion()`.

Save your program. Copy and paste the program code into **part 3(c)(i)** in the evidence document.

[1]

(ii) The class `TreasureChest` has a method `checkAnswer()` that takes the user's answer as a parameter. It returns `True` if the answer is correct and `False` otherwise.

Write the method `checkAnswer()`.

Save your program. Copy and paste the program code into **part 3(c)(ii)** in the evidence document.

[3]

(iii) The class `TreasureChest` has a method `getPoints()` that takes the number of attempts as a parameter.

- If the number of attempts is 1, it returns the value of `points`.
- If the number of attempts is 2, it returns the integer value of `points` divided by 2 (`DIV 2`).
- If the number of attempts is 3 or 4, it returns the integer value of `points` divided by 4 (`DIV 4`).
- If the number of attempts is not 1 or 2 or 3 or 4, it returns `0` (zero).

For example, a question is worth 100 points and the user took 2 attempts to give the correct answer. The user is awarded 50 points (100 DIV 2).

Write the method `getPoints()`.

Save your program. Copy and paste the program code into **part 3(c)(iii)** in the evidence document.

[5]

(iv) Write program code for the main program to:

- call the procedure `readData()`
- ask the user to enter a question number between 1 and 5
- output the question that matches the question number entered by the user
- check if the answer input by the user is correct using the method `checkAnswer()`
- repeat the question until the user inputs the correct answer
- count how many times the user attempted the question
- use the method `getPoints()` to return the number of points awarded
- output the number of points the user is awarded.

Save your program. Copy and paste the program code into **part 3(c)(iv)** in the evidence document.

[7]

(v) Test the program.

Take a screenshot showing the input(s) and output(s) for each of the following two tests.

In the first test:

- select question 1 and answer it correctly the first time.

In the second test:

- select question 5 and answer it correctly the second time.

Save your program. Copy and paste the screenshots into **part 3(c)(v)** in the evidence document.

[2]